

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY
POLITEHNICA BUCHAREST
FACULTY OF AUTOMATIC CONTROL AND COMPUTERS
COMPUTER SCIENCE AND ENGINEERING DEPARTMENT



DIPLOMA PROJECT

Improving Unikraft Support for Raspberry Pi

Octavian-Iulian Mihăilă

Thesis advisor

Conf.Dr.Ing. Adrian-Răzvan Deaconescu

BUCHAREST

2025

UNIVERSITATEA NAȚIONALĂ DE ȘTIINȚĂ ȘI TEHNOLOGIE
POLITEHNICA BUCUREȘTI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL DE CALCULATOARE



PROIECT DE DIPLOMĂ

Îmbunătățirea suportului Unikraft pentru Raspberry Pi

Octavian-Iulian Mihăilă

Coordonator științific

Conf.Dr.Ing. Adrian-Răzvan Deaconescu

BUCUREȘTI

2025

ABSTRACT

This thesis explores the use of unikernels - minimal, single-purpose operating systems designed for maximum efficiency and low overhead - on IoT specific hardware, deployed in a bare-metal manner. Unikraft - the development kit that allow for building this kind of unikernal applications with full control over the included components - is the targeted operating system for this study. The work focuses on improving Unikraft's support for the Raspberry Pi platform by enabling Symmetric Multi-processing (SMP), making it possible to fully utilize the device's multi-core capabilities. Further, the performance is evaluated, compared to the single core version of the platform, as cores are added the speed increases significantly, reaching 3.61x when all 4 cores are used. At the same time, a comparison with a minimal Linux distribution shows Unikraft's performance to be 20–30% lower due to differences in optimization.

SINOPSIS

Această teză explorează utilizarea unikernel-urilor (sisteme de operare minimale, cu un singur scop, concepute pentru o eficiență maximă și un overhead redus) pe hardware specific IoT, rulate într-o manieră bare-metal. Unikraft, kitul de dezvoltare care permite construirea acestui tip de aplicații unikernal, cu control deplin asupra componentelor incluse este sistemul de operare vizat pentru acest studiu. Lucrarea se concentrează pe îmbunătățirea suportului Unikraft pentru platforma Raspberry Pi prin activarea Symmetric Multi-processing (SMP), făcând posibilă utilizarea pe deplin a capacităților multi-core ale dispozitivului. În plus, performanța este evaluată, comparativ cu versiunea single core a platformei, pe măsură ce se adaugă nuclee viteza crește semnificativ, ajungând la 3,61x atunci când sunt folosite toate cele 4 nuclee. În același timp, o comparație cu o distribuție Linux minimă arată că performanța Unikraft este cu 20-30% mai mică din cauza diferențelor de optimizare.

CONTENTS

List of Abbreviations	3
1 Introduction	4
1.1 Context	4
1.2 Problem	5
1.3 Objectives and proposed solution	5
2 State of the Art and Related Work	6
2.1 State of the Art	6
2.2 Related work	7
3 Arhitectural Overview	8
3.1 Symmetric multiprocessing	8
3.2 Interrupt controller abstraction	9
3.3 Raspberry Pi 3 Model B hardware	10
4 Implementation Details	11
4.1 Interrupt controller refactor	11
4.2 Secondary cores bootstrapping	12
4.3 Unikraft SMP Integration	15
5 Benchmarks	18
5.1 Test environment description	18
5.2 Matrix multiplication	18
5.2.1 Description	18
5.2.2 Purpose	19
5.2.3 Results	19

5.3	Improved network application	20
5.3.1	Description	20
5.3.2	Purpose	20
5.3.3	Results	21
5.3.4	Analysis	22
5.4	Live core load	22
5.4.1	Description	22
5.4.2	Purpose	22
5.4.3	Results	23
6	Discussion - Future work	24
6.1	Parallelising NGINX on Unikraft/RPi	24
6.2	Extending support to newer Raspberry Pi models	25
6.3	Scheduler support	26
6.4	Other directions	26
7	Lessons learned	27
8	Conclusion	28
9	ARTIFACTS APPENDIX	31
9.1	Abstract	31
9.2	Hardware requirements & setup	31
9.3	Deploy Unikraft	31

LIST OF ABBREVIATIONS

SMP	Symmetric Multi-Processing
IRQ	Interrupt Request
FIQ	Fast Interrupt reQuest
ISR	Interrupt Service Routine
IPI	Inter-Processor Interrupt
GIC	Generic Interrupt Controller
APIC	Advanced Programmable Interrupt Controller
EL1/2	Exception Level 1/2 (ARM privilege level)
WFI	Wait For Interrupt
WFE	Wait For Event
FDT	Flattened Device Tree
MMU	Memory Management Unit
MMIO	Memory-Mapped I/O
PGD/PUD/PMD	Page-Global/Upper/Medium Directory, Page Table Entry
PTE	Page Table Entry
TTBR	Translation Table Base Register
SoC	System on a Chip
NIC	Network Interface Controller

1 INTRODUCTION

1.1 Context

Unikraft is a project developed to simplify the process of creating optimized applications that run directly on hardware or in virtualized environments, without the need for a full operating system. It allows the selection of only the components strictly necessary for an application, providing a customized, lightweight and resource-efficient system.

The concept behind Unikraft is that of unikernels – applications that are compiled together with the system kernel in a single image. These images can be executed directly on hardware or in a virtual environment, with the main benefits of a very small memory footprint, very fast boot times and a high level of isolation and security.

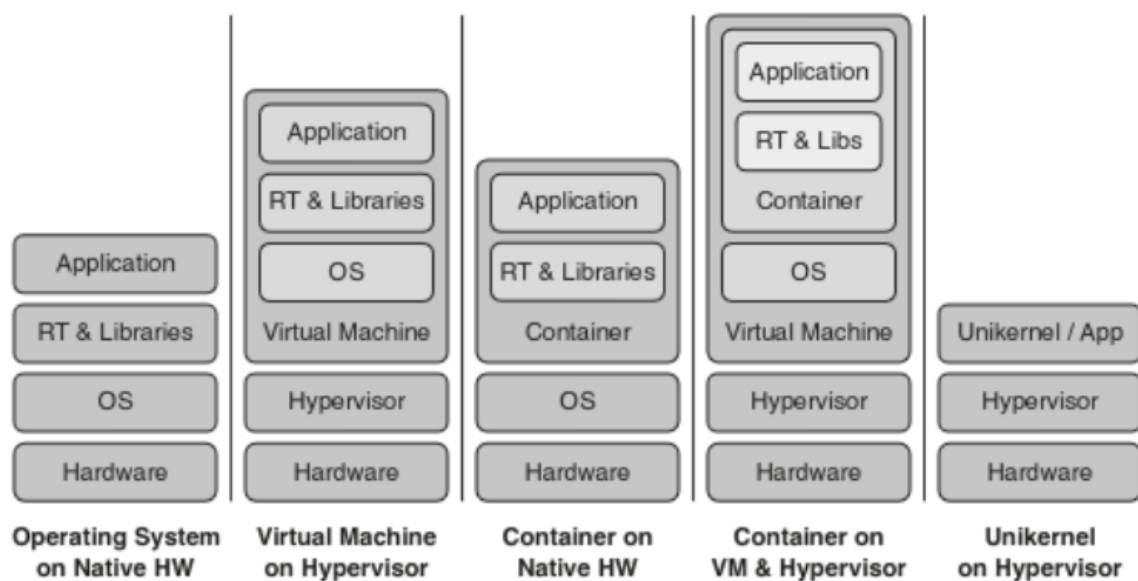


Figure 1: Software Stack Complexity Across Various Deployment Models. Adapted from [1]

1.2 Problem

Despite the existing support for SMP in Unikraft, this functionality is not available on the Raspberry Pi platform, which significantly limits the performance of applications running on this device. Currently, Unikraft runs on only a single processor core, even though the hardware allows parallel execution on multiple cores.

This limitation is a real problem, both in the context of the current platform, which has a network stack implementation constrained to running exclusively on a single core, and from the perspective of applications that could benefit from parallel execution, such as lightweight HTTP servers, real-time processing systems or applications that integrate concurrent components, where execution on a single core quickly becomes a bottleneck in performance and scalability.

Even with a fully functional SMP system, the absence of a scheduler still creates challenges: tasks have to be manually pinned to specific cores at the application level, and developers must handle all synchronization themselves. This manual approach is fragile, minimal changes in workload can leave one core saturated while the others sit idle, and every new feature forces a fresh round of placement tuning and race-condition checks. It also impacts portability: code that is written for Raspberry Pi 3B's four cores will need rework on a board with less or more CPUs. In practice, the effort spent babysitting core assignments can erase much of the simplicity that attracts people to unikernels in the first place.

1.3 Objectives and proposed solution

The main objective of this thesis is to enable support for multicore execution within Unikraft on the Raspberry Pi platform, to allow full use of available processing resources and to improve the performance of running applications. To achieve this objective, the project aims to analyze the existing implementation of SMP in Unikraft, identify limitations specific to the Raspberry Pi ARM architecture and develop the necessary mechanisms for multicore support. These include secondary CPU initialization and integration into SMP system, refactorization of the interrupt controller and MMU to make per cpu interrupts available. Validation of the functionality will be done through performance and stability tests, using real applications.

2 STATE OF THE ART AND RELATED WORK

2.1 State of the Art

Unikraft supports SMP in its core, but platform-specific integration is required to activate this functionality on bare-metal systems such as the Raspberry Pi 3 Model B — a strong candidate for such work given its widespread use in embedded systems and its quad-core ARM Cortex-A53 processor. The Raspberry Pi platform in Unikraft has seen several incremental improvements by various contributors, with more recent developments presented in the work of Job Paardekooper[2], who demonstrated the platform's feasibility for real-world applications by integrating a TCP/IP stack and evaluating networking performance against a minimal Linux distribution. However, as Job points out in his thesis, the lack of multi-core support was a major limitation in unlocking the full potential of the hardware for Unikraft. There are also other hardware-related limitations that affect the current Raspberry Pi platform, some of which are prerequisites for enabling multicore functionality.

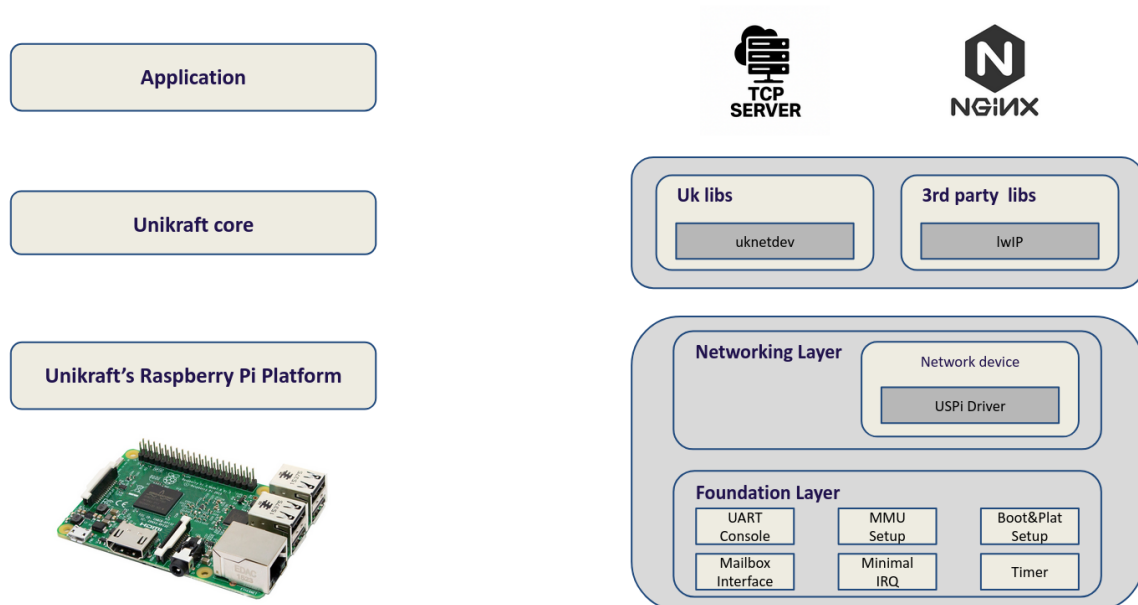


Figure 2: Unikraft on Raspberry Pi. Overview

2.2 Related work

With the idea of improving performance and handling workloads that benefit from parallel execution, several operating systems that take the form of a unikernel have begun to make efforts to include support for SMP. Projects like IncludeOS, MirageOS and OSv[3] have made noticeable progress in this direction, aiming to scale their systems across multiple cores, especially in cloud and virtualization environments where such support brings clear benefits.

Beyond cloud computing, another promising area for unikernels is embedded systems. This is especially true for IoT and edge computing, where low resource usage and direct hardware control are crucial. In this space, projects like MirageOS[4] and Nanos[5] have taken steps toward supporting the Raspberry Pi, a popular device due to its balance between cost and performance. Another project that addressed this idea is IncludeOS[6], which unlike the previously mentioned ones, did not intend to use a hypervisor, but to run bare-metal directly. However, there is no concrete information available whether this was implemented or not. Thus, Unikraft positions itself in this area as a system that can run bare-metal, already having support for Raspberry pi, but only in a single core manner.

3 ARCHITECTURAL OVERVIEW

3.1 Symmetric multiprocessing

Unikraft's SMP model simply treats each CPU core as a peer "logical CPU" running in a shared memory space. At boot, the primary core discovers and powers up all secondaries using some of the platform hooks, then each core runs a default entry function before sitting idle. From that point, applications may distribute workloads across the logical CPUs, for example, by dividing packet processing in parallel or running independent worker threads and coordinate interactions using standard synchronization primitives (mutexes, spinlocks, barriers, etc)[7]. Under the hood, each logical CPU also maintains a small, per-core function queue: when a task is dispatched to a core, it enqueues a callback in that queue and sends a run-IRQ to wake the core, which then pulls the next function off its queue and executes it. With per-CPU function queues and direct wake-up interrupts, each core can pull and execute tasks directly, without the need for a scheduler and expensive context switches.

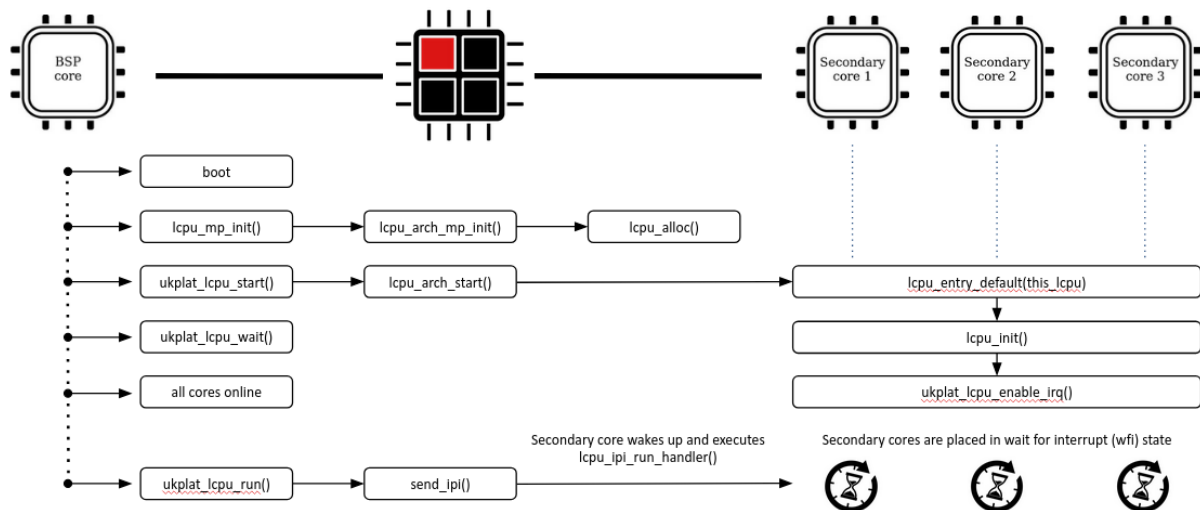


Figure 3: 4-core SMP execution flow. LCPU API description can be found in there: [8]

Once all four cores have been powered on and have run through the shared entry sequence (Figure 3), each logical CPU settles into the IDLE state, waiting for work to arrive. At this point the system itself is fully SMP-enabled: every core has a valid struct `lcpu`, its local interrupt context is configured, and it sits in a low-power wait-for-interrupt loop. From here, the application may enqueue a function on any core’s private queue and fire off a run-IRQ; the targeted core will immediately wake, pull its next ready task, and transition into a busy state to execute that function.

Figure 4 zooms in on that per-core lifecycle as a simple state machine. A core begins OFFLINE and moves to INIT when `lcpu_start()` kicks off its initialization. Upon successful setup (`lcpu_init()`), it enters IDLE, ready to accept dispatched tasks. Each `lcpu_run()` call pushes the core into BUSY0, and nested or additional calls drive it through BUSY1, BUSY2, ... up to BUSYN if needed. When functions complete, the core steps back down through the busy states to IDLE. An unrecoverable error at any point triggers `lcpu_halt()`, sending the core into the HALTED state until the next system reset.

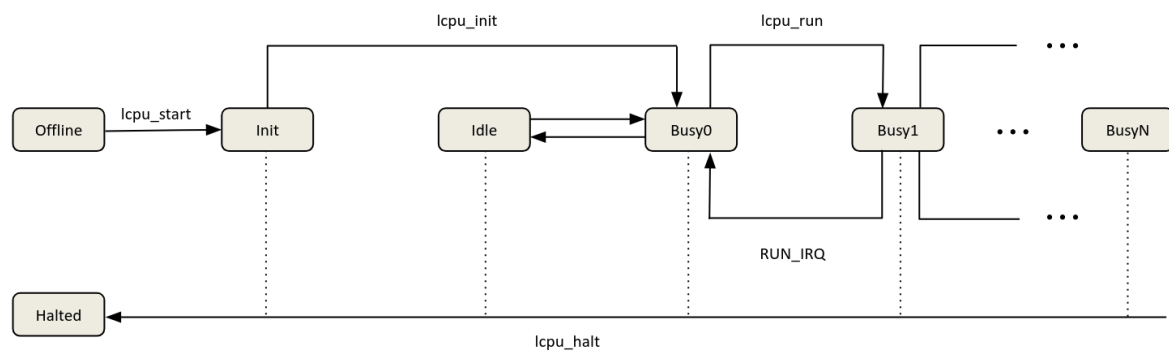


Figure 4: Logical CPU State Machine

3.2 Interrupt controller abstraction

Unikraft’s `intctlr` library provides a unified interface for handling interrupts across diverse hardware—whether an ARM GIC, an x86 APIC. During system initialization, the platform registers its controller by supplying a small set of callbacks for configuring IRQs, masking and unmasking lines, and translating device-tree entries into IRQ numbers. Once registered, `intctlr` maintains a global table of interrupt vectors and per-IRQ handler lists. This allows drivers for networking, timers, SMP bring-up, and others to work with interrupts without direct register manipulation: they simply request line allocation, configure triggers, and attach callbacks via `intctlr` library call.

At runtime, every interrupt is routed through a single dispatch path: the low-level ISR invokes `uk_intctlr_irq_handle()`, which emits a generic IRQ event and then iterates through the registered handlers until one claims responsibility. Behind the scenes, `intctlr` also supports bulk vector allocation, FDT “interrupts” property translation, and dynamic mask/unmask operations. All interrupt handling is done in one place, so Unikraft doesn’t depend on any specific controller. A new platform only needs a small piece of code to hook in its interrupt controller, and all existing drivers continue to work without change.

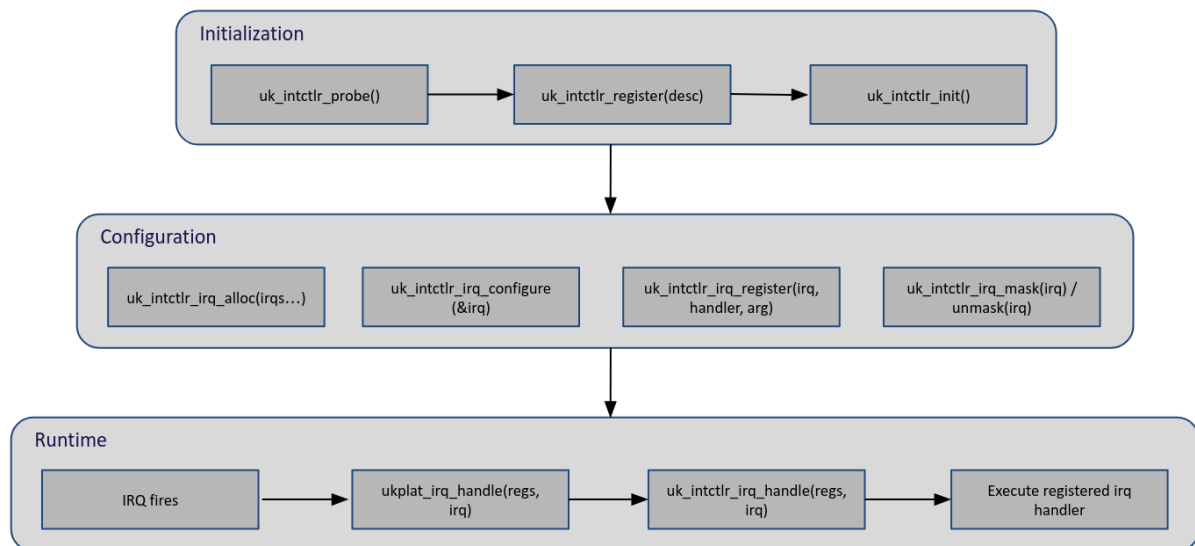


Figure 5: ukintctlr architecture overview

3.3 Raspberry Pi 3 Model B hardware

Interrupt delivery relies on a interrupt controller that is composed of two components. A global controller gathers the dozens of interrupt lines from GPIO, DMA, USB and other peripherals, assigns a priority, and forwards each request to the destination core’s local interrupt controller, that has independent IRQ and FIQ lines, along with a bank of per-core “mailboxes” that are used as inter-processor-interrupt (IPI) mechanism.

Raspberry Pi 3B’s Ethernet jack is not wired to a native MAC; instead, it is connected to the LAN9514 chip, which is both a USB 2.0 hub and a 10/100 Mb Ethernet controller. At boot the VideoCore firmware turns on the SoC’s single USB-host port, discovers the LAN9514, and exposes it to ARM software as an ordinary USB network device, then all the frames will move through USB bulk endpoints, sharing the same 480 Mb/s link with any other type-A connected device.

4 IMPLEMENTATION DETAILS

4.1 Interrupt controller refactor

The existing Raspberry Pi platform driver only covered the global interrupt controller and completely bypassed Unikraft's ukintctlr interface. Since SMP system depends on each core's local interrupt controller (to send IPIs when tasks are dispatched to secondaries), first thing that had to be done was to allocate extra virtual memory for the area that covers this peripheral. In the boot-time page-table setup, reserved a second PUD entry and installed a single 2 MiB section at 0x40000000–0x4001FFFF, mapping the core-private mailbox, priority and IPI registers as device memory. With those local-INTC registers now behind the MMU, the ukintctlr layer can uniformly configure, mask/unmask, and dispatch both global and per-core interrupts without poking physical addresses directly.

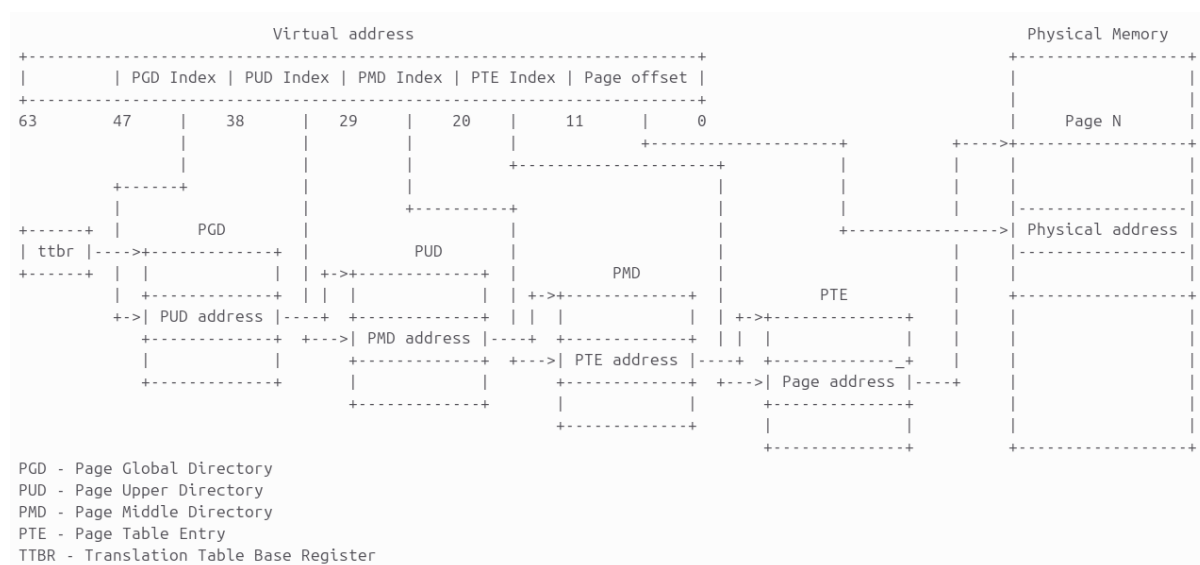


Figure 6: Memory translation process. Adapted from: [10]

Originally, the Pi's MMU setup used a single PGD entry and a single PUD entry, with one PMD page mapping the entire 1 GiB of RAM via 2 MiB sections—no PTEs and just three 4 KiB pages in total (one each for PGD, PUD, and PMD). To expose the core-private interrupt controller registers, we added a second PUD slot (PUD[1]) and gave it its own PMD page. That new PMD then holds a single 2 MiB device-memory section at 0x4000 0000, covering all local-INTC mailboxes, priority and IPI registers in one descriptor. By sticking with section mappings, we eliminate per-page overhead, keep the hierarchy at two levels, confine all MMIO to its own PMD page, and leave the original RAM mapping completely untouched.

The Raspberry Pi port now feeds both its global interrupt lines and its core-private mailbox IPs into Unikraft's common interrupt api (ukintctlr), just like GIC or APIC reference drivers from unikraft do, providing all the functionality that is needed for running in a multicore manner.

4.2 Secondary cores bootstrapping

Raspberry Pi's stock boot arrangement wakes every Cortex-A53 in EL2 but relies on the operating system to decide what to do with the three secondary cores. Unikraft's original start.S simply froze any secondary cpu, which was acceptable when not using SMP. Obviously, the bring up for the remaining cores is vital if we intend to run in multicore, so significant changes were needed in the boot flow. The secondary-CPU initialization along with park-and-spin logic is inspired from the multicore bootstrap written by Leon de Beer[11] for his real time OS that runs on the same hardware.

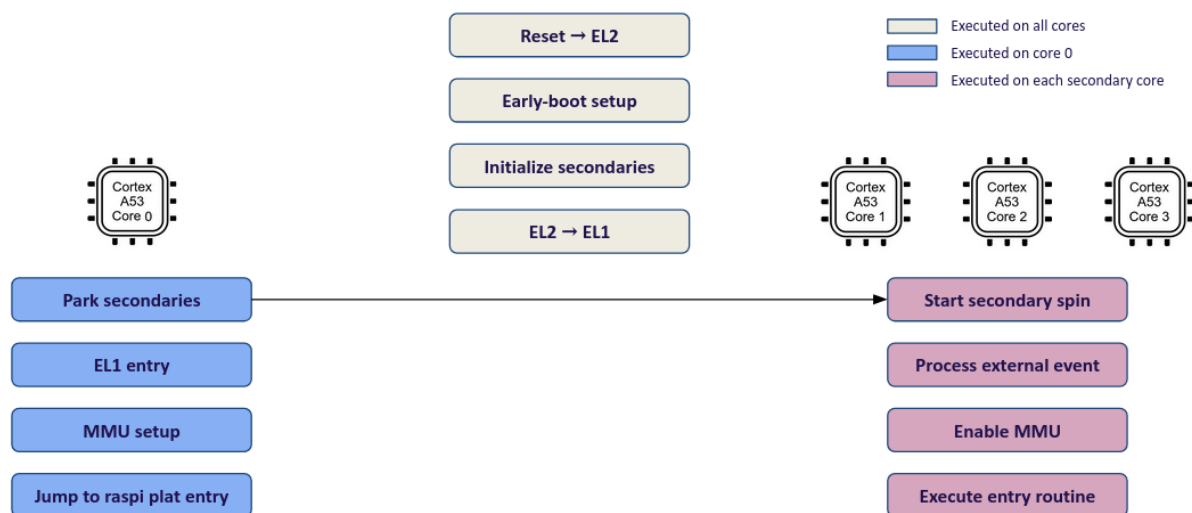


Figure 7: Multicore bootstrap on Raspberry Pi 3B

In this new version of the boot file, all four cores first execute a shared “early” section. During this stage they claim a private stack for each exception level, record a handful of board constants (peripheral base, boot address, build flags). After that, basic traps are disabled, the generic timer is opened for EL1 access, and diverse system-control and hypervisor-configuration registers are written in order to finish core initialization and prepare to jump into EL1. Next, it loads the address of the interrupt vector table, sets the CPU state that should be restored, and issues one `eret` instruction. That single instruction switches every core from EL2 to EL1, leaving them there with interrupts still off and their stacks already set up.

Once in EL1 the flow forks, Core 0 clears the BSS, constructs the kernel page tables, enables caches, configures MMU and “parks” the other three into a spin loop, which is written in a manner that makes it possible for the operating system, initially running on core zero, to notify the secondaries. This is done via mailboxes in which the address for the first executed routine is written. This routine is usually the entry point for the core into the multicore environment, in our scenario the SMP system. Also, when such a routine has to be executed by a core that was just released from spinning, the page tables that created by core 0 are loaded and the secondary perform necessary configurations. Thus there is no need to rebuild page tables or duplicate global initialisation work in order to acces memory corectly from anywhere.

In addition to a plain function pointer, the spin-loop can hand the secondary core a pointer to a parameter block. Because mailbox-1 and mailbox-2 are treated as the low and high halves of a 64-bit value, core 0 can pass the address of any structure that contains a full set of arguments or per-CPU context—e.g. `lcpu_entry_default` receive logical cpu structure as paramter. The awakened core rebuilds that 64-bit pointer, flushes the mailbox so the slot is free for future work, and start executing the target function it received.

```

1 .equ spin_cpu1, 0xe0
2     mov x1, #spin_cpu1      // Spin core1 jump address
3     ldr x2, =multicore_start // Function we are going to call
4     str x2, [x1]            // Store the function address
5     sev                     // Wake core1 up
6     ldr x3, =RPi_CoresReady // Set CoresReady count address
7 .WaitCore1ACK:
8     ldr w1, [x3]            // Read CoresReady count
9     cmp w1, #2              // Wait for 2nd core to get ready
10    bne .WaitCore1ACK       // Core1 not ready so read again
11 .equ spin_cpu2, 0xe8
12    mov x1, #spin_cpu2      // Spin core2 jump address
13    ldr x2, =multicore_start // Function we are going to call
14    str x2, [x1]            // Store the function address
15    sev                     // Wake core2 up
16    ldr x3, =RPi_CoresReady // Set CoresReady count address
17 .WaitCore2ACK:
18    ldr w1, [x3]            // Read CoresReady count
19    cmp w1, #3              // Wait for 3rd core to get ready
20    bne .WaitCore2ACK       // Core2 not ready so read again
21 .equ spin_cpu3, 0xf0
22    mov x1, #spin_cpu3      // Spin core3 jump address
23    ldr x2, =multicore_start // Function we are going to call
24    str x2, [x1]            // Store the function address
25    sev                     // Wake core3 up
26    ldr x3, =RPi_CoresReady // Set CoresReady count address
27 .WaitCore3ACK:

```

```

28     ldr    w1, [x3]           // Read CoresReady count
29     cmp    w1, #4             // Wait for 4th core to get ready
30     bne    .WaitCore3ACK      // Core3 not ready so read again

```

Listing 4.1: Core 0 initializes and wakes up secondary cores (1–3) by writing the entry-point address (multicore_start) into each core’s mailbox slot (spin_cpu), issuing a SEV event, and then polling the shared RPi_CoresReady counter until each one acknowledges it is ready.

```

1     .balign 4
2 StartSecondarySpin:
3     wfe
4
5 .spin:
6     // Compute this core's mailbox-0_RD base in x7
7     mov     x5, #192
8     movk    x5, #0x4000, lsl #16 // x5 = 0x4000_00C0
9     mrs     x6, MPIDR_EL1
10    and     x6, x6, #0x3
11    ubfiz   x6, x6, #4, #4 // x6 = core_id<<4
12    add     x7, x5, x6 // x7 = mailbox-0_RD for this core
13
14    // Fetch fn
15    ldr     w4, [x7]
16    cbz     w4, StartSecondarySpin
17
18    // Rebuild the 64-bit argument
19    ldr     w0, [x7, #4] // mailbox-1_RD      low half
20    ldr     w1, [x7, #8] // mailbox-2_RD      high half
21    orr     x0, x0, x1, lsl #32
22
23    // clear all four slots
24    str     w4, [x7]
25    str     w0, [x7, #4]
26    str     w1, [x7, #8]
27    str     w2, [x7, #12]
28
29    mov     x28, x4 // save the function pointer
30    mov     x29, x0 // save the argument low 64 bits
31    mov     x30, x1 // save the argument high 64 bits (if any)
32
33    bl     enable_secondary_mmu
34
35    // Restore the argument exactly as the caller expects
36    mov     x0, x29

```

```

37     mov    x1, x30
38     br     x28 // jump to fn(arg) with MMU & caches ON
39
40     // Never returns
41     b      StartSecondarySpin

```

Listing 4.2: After waking from WFE, each core computes its mailbox address, loads the startup function pointer and 64-bit argument from it, clears the mailbox slots, calls `enable_secondary_mmu` to turn on its MMU, then branches to the function entry point.

4.3 Unikraft SMP Integration

After completing the board-specific work needed for Unikraft to run, but before stepping into the actual kernel, registration for the two interrupt lines, RUN and WAKEUP is done, lines that will be later used by the platform to handle IPIs. Along with that `ukplat_lcpu_start` is invoked, the generic routine that brings the secondary cores into SMP system.

During SMP initialization, the platform-specific hook `lcpu_arch_start` activates each core. It sends a function pointer and the logical-CPU argument through the mailbox interface to the local interrupt controller, breaking the core out of its spin loop and jumping to the entry routine. The core then enters WFI state, waiting for further work delivered via IPIs, generated by later calls to `lcpu_arch_run`, the usual application-level mechanism for scheduling tasks on the SMP-managed cores. Figure 10 presents all the mailboxes from the local interrupt controller used by the raspberry pi platform for dealing with IPIs:

REGISTER	BASE OFFSET	CORE N ADDRESS	PURPOSE	MASK/BIT
MBOX_IRQCNTL	$0x50 + 4 \times N$	$0x40000000 + (0x50 + 4 \times N)$	Mask/unmask mailbox interrupt	$MBOX0_IRQ = 1 < < 0$
IRQ_SOURCE	$0x60 + 4 \times N$	$0x40000000 + (0x60 + 4 \times N)$	Read pending source bits (IPI pending?)	$INT_SRC_MBOX0 = 1 < < 4$
MBO_SET	$0x80 + 0x10 \times N$	$0x40000000 + (0x80 + 0x10 \times N)$	Spin-table: write low 32 b of the entry-point	
MB1_SET	$0x84 + 0x10 \times N$	$0x40000000 + (0x84 + 0x10 \times N)$	Spin-table: write low 32 b of the 64-bit argument	
MB2_SET	$0x88 + 0x10 \times N$	$0x40000000 + (0x88 + 0x10 \times N)$	Spin-table: write high 32 b of the argument	
MBO_RDCLR	$0xC0 + 0x10 \times N$	$0x40000000 + (0xC0 + 0x10 \times N)$	Spin-table: read & clear; non-zero → core wakes up	

Figure 8: Per-Core Mailbox Registers[12]

```

1 void start_on_core(unsigned core, void (*fn)(void *),
2                     void *arg)
3 {
4     uint64_t a = (uint64_t)(uintptr_t)arg;
5
6     DataSyncBarrier();
7
8     /* Split the 64-bit argument across two 32-bit mailboxes */
9     *(volatile uint32_t *)A53_MB1(core) =
10         (uint32_t)(a & 0xFFFFFFFF);
11     *(volatile uint32_t *)A53_MB2(core) = (uint32_t)(a >> 32);
12
13     /* Poke the fn-pointer into mailbox-0 (low 32 bits only) */
14     *(volatile uint32_t *)A53_MB0(core) =
15         (uint32_t)(uintptr_t)fn;
16
17     DataSyncBarrier();
18     SendEvent();
19 }
20
21 static inline void flush_to_ram(const void *addr, size_t len)
22 {
23     uintptr_t p = (uintptr_t)addr & ~(CACHE_LINE_SIZE-1);
24     uintptr_t end = (uintptr_t)addr + len;
25
26     for (; p < end; p += CACHE_LINE_SIZE)
27         __asm__ volatile("dc civac, %0" :: "r"(p) : "memory");
28
29     DataSyncBarrier();
30 }
31
32 int lcpu_arch_mp_init(void *arg)
33 {
34     for (__lcpuid id = 1;
35         id < CONFIG_UKPLAT_LCPU_MAXCOUNT; id++) {
36         struct lcpu *lc = lcpu_alloc(id);
37     }
38
39     /* Make sure all cores see the new config */
40     DataSyncBarrier();
41
42     return 0;
43 }
44

```

```

45 int lcpu_arch_start(struct lcpu *lcpu, unsigned long flags) {
46     // Before starting the core, put everything in order.
47     flush_to_ram(lcpu, sizeof(*lcpu));
48
49     start_on_core(lcpu->id, lcpu_entry_default, lcpu);
50
51     return 0;
52 }
53
54 int lcpu_arch_run(struct lcpu *lcpu,
55                  const struct ukplat_lcpu_func *fn,
56                  unsigned long flags)
57 {
58     int rc = lcpu_fn_enqueue(lcpu, fn);
59     if (rc) return rc;
60
61     // Make sure the fn object is visible before sending IPI
62     DataSyncBarrier();
63
64     send_ipi(lcpu->id);
65
66     return 0;
67 }

```

Listing 4.3: Part of the SMP API implementation

5 BENCHMARKS

5.1 Test environment description

The Unikraft platform for Raspberry Pi offers the ability to run routines on specific cores, from the application level through the SMP API present in the operating system. Synchronization is managed entirely by the developer, using SMP-safe primitives at the application level, rather than via any automatic runtime coordination. Time measurements are made by the main core, upon completion of execution by the secondary cores using the internal monotonic clock (`ukplat_monotonic_clock`) which provides high resolution.

Raspberry Pi OS Lite runs on an ARMv8 optimized Linux 5.x kernel, with CFS scheduler that dynamically balances the workloads between available cores. For multicore tests we use `pthread_create()` at the application level while timing measurements are performed with `clock_gettime(CLOCK_MONOTONIC)`. The number of active cores is set at boot time by the `maxcpus` parameter in `cmdline.txt`, without any additional software mechanism, accurately reflecting the hardware configuration defined at startup.

The Raspberry Pi 3 Model B[9] is built around Broadcom's BCM2837 SoC, featuring a quad-core ARM Cortex-A53 cluster at 1.2 GHz. Each core has private 32 KiB L1 instruction and data caches, backed by a shared 512 KiB L2. The main memory is 1GB LPDDR2 SDRAM, operating at 900 MHz Raspberry Pi accessed via an external data bus, which provides a theoretical bandwidth of up to 6.4 GB/s.

5.2 Matrix multiplication

5.2.1 Description

The benchmark multiplies two 512×512 matrices by partitioning the row range equally across all active cores. On Unikraft, this is scheduled on secondary cores via `ukplat_lcpu_run()`, while core 0 does its slice with `run_core_zero_share()` and then waits for the others by executing `ukplat_lcpu_wait()`. In Linux, threads are spawned with `pthread_create()` and set CPU affinity (`pthread_setaffinity_np()`) so each thread handles a contiguous block of rows.

5.2.2 Purpose

Proof of SMP: Demonstrate that porting and enabling SMP support on the Unikraft platform for Raspberry Pi works correctly, by dividing the workload between cores and obtaining consistent results.

Linux comparison: Unikraft SMP performance compared to a reference system like Raspberry Pi OS Lite

Scalability evaluation: Measure how much execution speeds up when running on 2, 3, and 4 cores, compared to a single core.

5.2.3 Results

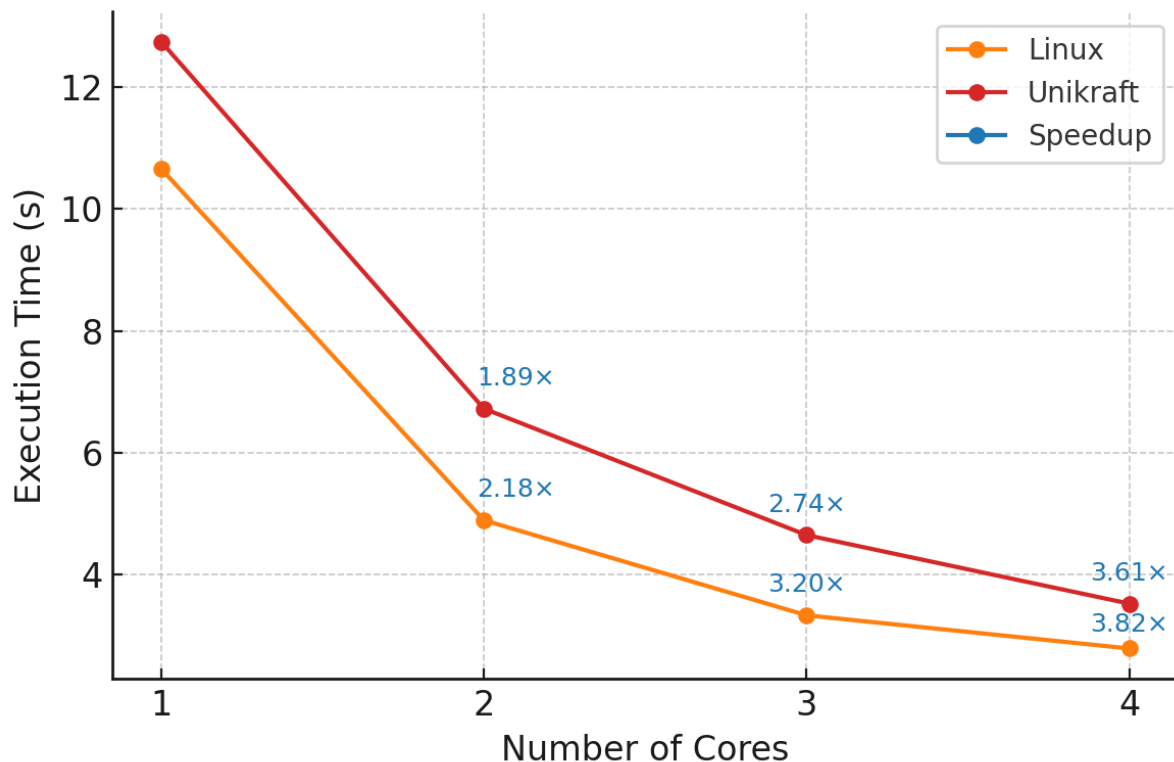


Figure 9: Matrix Multiplication on Raspberry Pi: Unikraft Linux

We observe that the introduction of a second core reduces the execution time by almost half, signaling a very good parallelization potential and implicitly the fact that the operations do not depend on each other. As we add the third and fourth cores, however, the gain decreases progressively, indicating the presence of a small serial portion and costs that are most likely related to the competition for the bandwidth of the caches and shared memory.

Linux achieves approximately 20–30% better times across all configurations, due to lower thread startup and scheduling costs (pthread), compared to SMP mechanisms that have significant overhead for starting task execution. Additionally, Linux is a mature system with decades of optimizations for scheduling, memory management, and drivers, which ensures superior performance in multicore scenarios.

5.3 Improved network application

5.3.1 Description

This benchmark is based on a client-server application, with some CPU-intensive tasks specific to network communication. A compression algorithm (LZ77), which scans the data window to replace repeated sequences with shorter references, a block cipher (TEA) with multiple processing rounds to simulate real confidentiality requirements and finally, before sending the data, a CRC-32 checksum is calculated to detect possible packet corruption.

In SMP mode, the 1 MB buffer is divided into four equal slices that are processed in parallel: three slices on the secondary cores via `ukplat_lcpu_run()`, and one on the main core via `run_bsp_slice()`. Each slice undergoes LZ77 compression, TEA encryption, and 8-byte alignment, then the results are combined into a single buffer. Before transmission, the main core calculates a CRC-32 on the entire encrypted area and prepares the network packet with the structure: 4 bytes CRC (network-order), 4 bytes payload length, and then the compressed encrypted data (1 MB plus padding). In single-core mode, all stages are executed sequentially on the same core, without using the SMP dispatch mechanism

5.3.2 Purpose

In his thesis Job Paardekooper evaluates the pure performance of network communication, but in most real-world situations, an application will also perform various CPU-intensive tasks. The purpose of this benchmark is to evaluate multicore performance using SMP in a more complex scenario (with repetitive tasks scheduled across multiple cores, shared-memory interactions and cache-coherence traffic), but also to see how we can parallelize a client-server application using this system and what challenges may arise.

5.3.3 Results

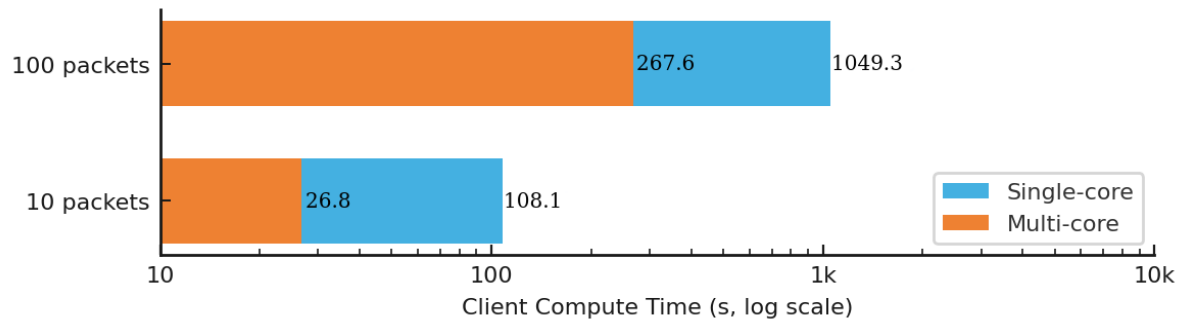


Figure 10

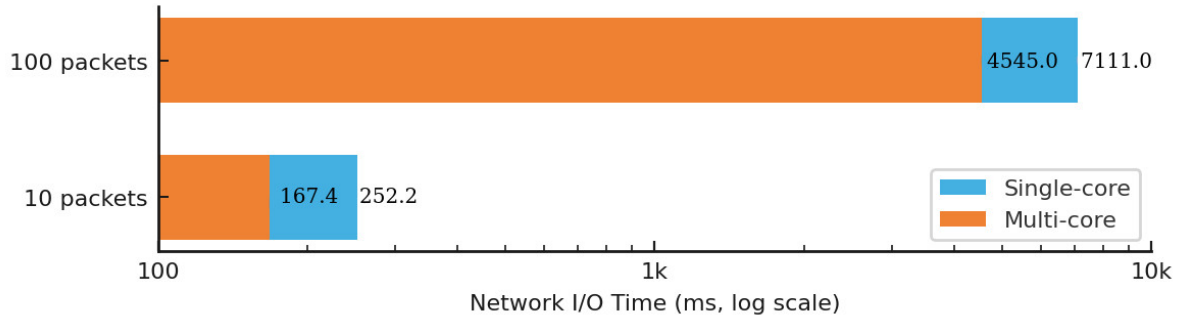


Figure 11

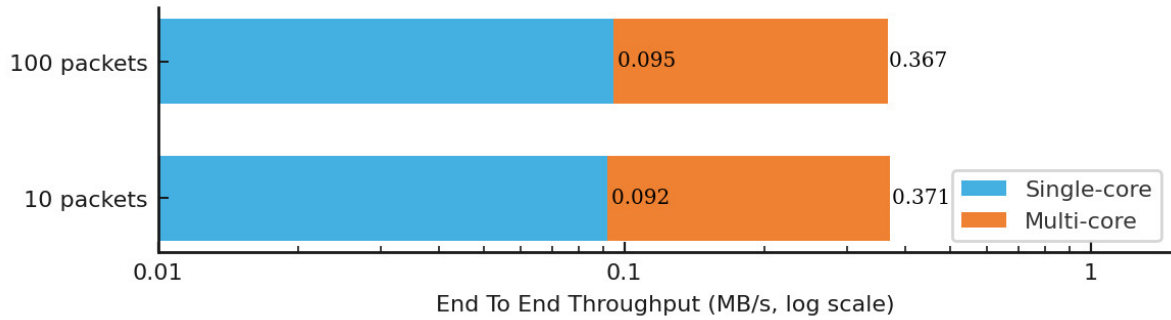


Figure 12

As can be seen in the graphs, adding CPU-intensive tasks means that when running on a single core, the entire network capacity cannot be used to its full potential, because data is sent much less frequently, the core is involved most of the time in processing for compression and encryption. Another interesting behavior observed is that the duration for network I/O increases with time, something that Job also observed in his evaluations and which requires further investigation. For now, the reduction in execution time brought by using all available cores masks these issues, allowing for increased edge computing capability.

5.3.4 Analysis

Beyond the single-client scenario, SMP on Unikraft/RPI could be extended to support multiple concurrent clients by dedicating one core exclusively to network I/O and connection management, while exposing the remaining cores as a shared pool that each client can borrow for CPU-intensive preprocessing. In practice, this might involve pinning the network interrupt queue and `accept()` loop to core 0, and scheduling each incoming client's intensive computation onto worker cores via some sort of task queue. Such an approach would improve isolation and bring a significant benefit from load balancing, while having many cores touch shared buffers increases cache-coherence traffic and demand for scalable synchronization mechanisms at the application level.

5.4 Live core load

5.4.1 Description

For this benchmark, a image-processing algorithm (marching squares) is deployed where each core is executin some intensive computational tasks for a different picture. Constantly samples are taken to see the CPU load, especially in areas with large loops and intensive mathematical operations. Only the secondary cores are used for processing, core 0 is involved in scheduling activities, logging and network I/O, including sending processed images to the server.

CPU load quantifies the proportion of real time during which a processor executes instructions. It is obtained by sampling the architectural cycle counter (`PMCCNTR_EL0`), which increments once per core clock tick, and dividing the elapsed ticks by the counter-frequency register (`CNTFRQ_EL0`). Averaging these per-core ratios over successive windows provides a stable measure of system utilisation. A system monitor typically averages many such per-core samples to smooth out bursts and present a stable utilisation percentage, but we are not particularly interested in that, since this is just a proof of concept for a more complex unikernal app that uses SMP.

5.4.2 Purpose

The goal is to assess how the Raspberry Pi's SMP architecture handles a heterogeneous, core-pinned workload by observing live CPU load on each core. Tracking these metrics helps identify performance bottlenecks, verify effective load balancing, and demonstrate the responsiveness and scalability of the system under realistic multitasking conditions.

5.4.3 Results

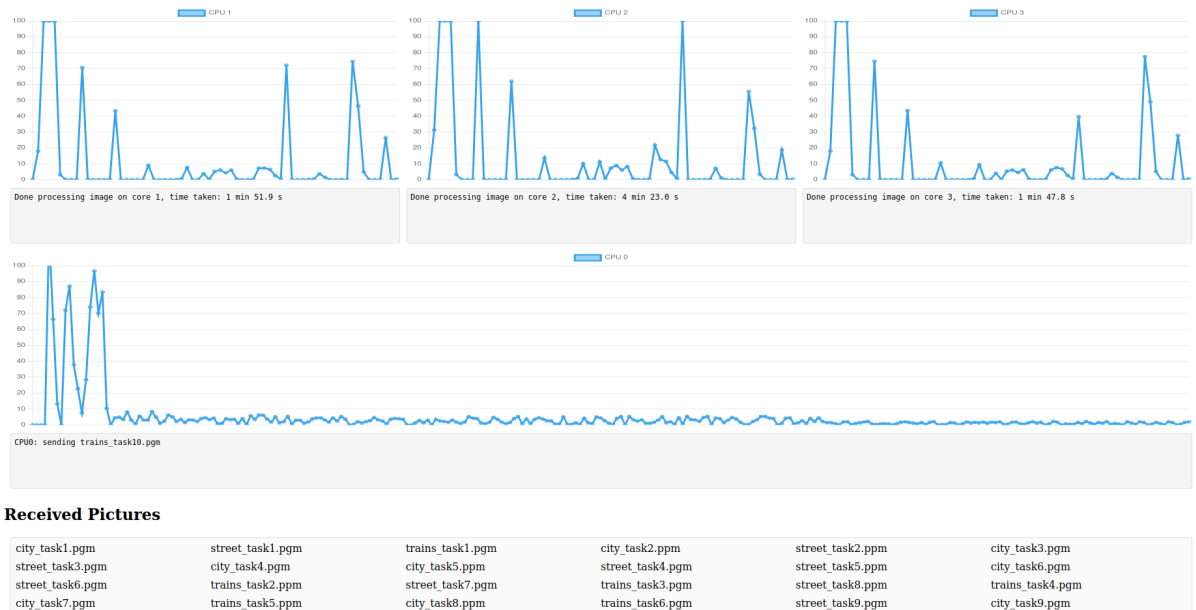


Figure 13: Parallel Image processing using Unikraft SMP

As shown in the graphs, there are some peaks which represent samples for each task, collected in the computationally intensive areas. The same tasks are runned on each core, but with different input, which makes slight differences visible in terms of load, but especially in execution time, differences that arise due to the image dimensions. At the same time core 0 experiences heavy computing only at the time when it loads the images from the temporary filesystem (initrd). The rest of the operations seems to not request too much CPU, which means that in this scenario with small size images that don't take long to send, core 0 might be suitable for executing some tasks as well, achieving better performance.

This benchmark should be viewed as a proof-of-concept tracing mechanism. Its value lies in showing how lightweight cycle-counter sampling can expose per-core behaviour in real time, making it possible to do instrumentation for the future scheduler and other SMP related functionality.

6 DISCUSSION - FUTURE WORK

Currently, the unikraft platform for Raspberry Pi is reaching the point where it can be successfully used to deploy basic IoT applications, as evidenced by the benchmarks performed by Job on the networking side and those for edge computing presented previously. With a baseline established, the project can turn from “prove it works” to “make it versatile”. What follows describes the most promising directions for that next phase of work, mainly from the perspective of parallelization brought by SMP functionality.

6.1 Parallelising NGINX on Unikraft/RPi

At launch the master process parses the configuration file, then forks N identical worker processes. The master’s job ends there, it never touches client traffic again. Each worker is a single-threaded, event-driven loop that opens its own listening socket. When a SYN arrives the operating system decides which socket, and implicitly which worker, receives the new connection. With a modern kernel and reuseport enabled, connections are distributed evenly; without it, the accept-mutex lets only one worker at a time call `accept()`. Once a worker owns the connection it handles the entire request/response lifecycle without further synchronisation.

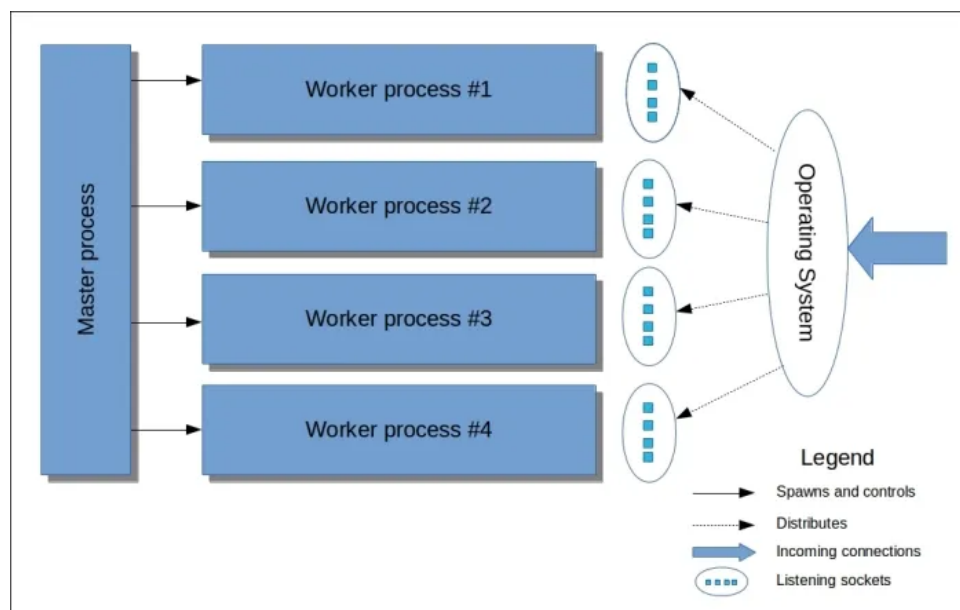


Figure 14: NGINX multi-worker model. Adapted from [13]

An attempt to parallelize the nginx application involved starting 4 workers by directly modifying the nginx code and pinning each one to a core, but the results were poor: they all listen on the same TCP port, and in the current configuration of the Ethernet driver, the NIC interrupt only reaches core 0; only the worker on that core can `accept()`, the other three remain idle. A possible solution to this limitation would be to extend the ability to catch interrupts to the secondary cores, but this involves several changes, since a secondary core, once removed from the SMP execution context, would also have to be brought back there to the state it was in before.

Another cleaner approach could be to have a minimal scheduler functionality that would distribute worker processes to free cores only when needed (the worker has a request to service). Another possibility worth exploring is to intercept `fork()`, which NGINX uses to launch workers, and replace it with a function that spawns a process in which the worker runs on a specific core. In this way, the NGINX code would remain unchanged and the placement logic would be moved to the scheduler level.

6.2 Extending support to newer Raspberry Pi models

For now, the platform is compatible only with the Raspberry Pi 3B, but with minimal modifications it should also be able to run on the 2B model, the board being largely similar. A more significant effort should be made to support newer versions, such as 4 and 5, which come with an imminent benefit, the fact that they have an interrupt controller that is compliant with the arm GICv2 standard. There is already a driver implementation for this standard within unikraft, which would make porting the SMP system to these devices much easier.

In order to be able to extend the platform for other boards in a generic, easy to maintain, and extendable manner, a device tree blob-based approach is suitable to access information about peripherals and board specifications via MMIO, instead of relying on hardcoded addresses to interact with the hardware. An attempt for this was made at the beginning of the integration work, with the idea of detecting the cores in a dynamic way, but it was later abandoned, mainly due to the fact that the secondary cores bringup is done directly from the code written in assembly, an inappropriate place to interact with the libfdt library from unikraft, which is responsible for operations on the device tree. In order to be able to make such a transition for the entire platform, a more detailed analysis is necessary that takes into account several components involved, especially drivers.

Other aspects to consider for this porting effort are that the newer versions have more RAM—information available in the DTB—but the code responsible for MMU initialization must be refactored so that it can handle this dynamically. Another issue is the network driver: it is incompatible with the newer boards for hardware reasons. On the 3 B, networking is provided by the USB-based LAN9514 10/100 Mbps controller, whereas the newer models use a native Gigabit MAC and PHY wired directly to the SoC, so another driver port is required.

6.3 Scheduler support

Unikraft's core provides an SMP API, but it does not yet include a general multi-core scheduler. In the current model, each logical CPU exposes a per-CPU function queue and relies on the application to explicitly dispatch work onto secondary cores via `ukplat_lcpu_run()` and related calls as previously describe in the arhitectural overview section.

Because there is no built-in scheduler to balance or preempt tasks, Unikraft currently requires developers to handle load distribution and synchronization entirely at the application level. This "manual pinning" approach can work for simple workloads, like the ones used for benchmarking, but it gets harder for something more complex, where threads may need to share work, react to changing workloads, or contend for shared resources. Moreover, even though sometimes such an approach with maximum control for the developer can be useful, in most cases it leads to a lack of load balancing between cores or to cases where one CPU can starve other threads on the same core, limiting fairness and responsiveness.

One possbile aproach would be to enable a hardware timer on each core so that every few milliseconds an interrupt fires and to maintain a small run list of threads per LCPU. When a thread's time slice ends or it blocks, it is moved to the tail of its core's list and the next thread is picked to run via `lcpu_fn_enqueue()`. New threads can go into the least loaded core list, and if one runs out of work it steals threads from another before idling. This simple setup gives round-robin fairness and dynamic load balancing across all cores, but it still incurs overhead from frequent timer interrupts, context switches, and the synchronization needed for the queues.

Another option would be to modify the cooperative scheduler already present in unikraft so that it can work with multiple cores simultaneously through a thread that periodically polls the queues of each `lcpu`, and if such a queue exceeds a certain threshold, the threads are migrated to a less load one. This can be done directly from the scheduler code[15], using the SMP API, but it involves an in depth analysis, not only from the perspective of the Raspberry PI platform but also of the others that the operating system supports.

6.4 Other directions

One notable shortcoming of the Unikraft/Raspberry Pi platform is the lack of drivers for many peripherals that are essential in order to transform this prototype into a versatile IoT suitable platform. Adding support for CSI cameras, HDMI/DSI displays, GPIO, I²C, and SPI modules would immediately unlock common use cases like video capture, sensor interfacing and other useful things. Many of those drivers can be ported from license compatible projects such as FreeBSD or Zephyr.

Another interesting direction would be to build a lightweight inter-core messaging layer on top of the Pi's mailbox hardware using per-core ring buffers (in shared memory) and mailboxes to notify CPUs of new messages. This would be helpful for both implementing a dynamic scheduler in which cores can push or steal work directly via message queues, balancing load at runtime or for certain security-related features, such as isolating untrusted workloads to specific core domains.

7 LESSONS LEARNED

While working on the functions that are needed by Unikraft to initialize secondary LCPUs and bring them into SMP system, I ran into several problems when it came to executing code on cores just removed from the spin loop, due to the difference between the data in the cache and the data in RAM. Specifically, without flushing the LCPU structure to RAM (`dc civac`) and issuing a `DataSyncBarrier()`, the secondary core could read invalid pointers. Similarly, mailbox writes must be ordered: placing a `DataSyncBarrier()` before `SendEvent()` ensures both the function pointer and its argument are visible on the other core. These are important things to keep in mind when working on improving this SMP integration in order to make sure that all shared data and control signals are properly synchronized across cores.

In the boot part, I've encountered some issues that were related to the fact that the exception vector table and irq handler stub were not placed at the required alignment, causing IRQs to not execute their associated handler. It turned out that forgetting the proper `.balign` resulted in the cpu starting to executed from garbage section, so care must be taken that these sections are in accordance with the ARM standard, otherwise silent break at boot time can occur.

Another aspect to keep in mind is that when platform-level code runs on a core outside the SMP system (for example, in an interrupt handler), the handler must ensure that the core is returned to WFI and its state is updated in the SMP subsystem. Otherwise, Unikraft will no longer be able to use it, and it will become a dead core from the perspective of the operating system and, implicitly, the application.

8 CONCLUSION

This thesis demonstrates that Unikraft can fully exploit the potential of the Raspberry Pi 3 Model B hardware by enabling multicore execution in a bare-metal manner, taking advantage of the SMP mechanisms already present in the operating system. Experimental results show that for a simple task such as matrix multiplication, a speedup of 3.61x is obtained, which places unikraft in a performance area close to Linux, being only 20-30% lower, which is very good considering the lack of optimizations.

Now having the ability to do edge computing with decent performance, together with the already existing networking stack and the benefits of a unikernel-based system, makes this platform a feasible option for application developers for the IoT area. However, there are still many things that can be improved on this multi-core side to make it more attractive and developer-friendly, directions analyzed and presented previously, both in terms of improving performance and usability.

BIBLIOGRAPHY

- [1] *A Performance Survey of Lightweight Virtualization Techniques*
https://www.researchgate.net/publication/319407909_A_Performance_Survey_of_Lightweight_Virtualization_Techniques
- [2] *Using the Unikernel Development Kit “Unikraft” to Develop Networked Embedded Systems Applications*
<https://github.com/jobpaardekooper/unikraft-rpi/blob/measurements/measurements/thesis.pdf>
- [3] *Universal Unikernel – OSv*
<https://mikelangelo-project.eu/technology/universal-unikernel-osv/>
- [4] *MirageOS on ARM64*
<https://mirage.io/docs/arm64>
- [5] *Nanos on 64-bit ARM Platforms*
<https://nanovms.com/dev/tutorials/nanos-on-64-bit-arm>
- [6] *IncludeOS: A Library Operating System for IoT and Cloud Services*
<https://www.includeos.org/technology.html>
- [7] *Adding SMP Support*
<https://unikraft.org/blog/2022-06-27-unikraft-synchronization>
- [8] *Unikraft Releases v0.9.0 (Hyperion)*
<https://unikraft.org/blog/2022-06-13-unikraft-releases-hyperion>
- [9] *Raspberry Pi 3: Specs, Benchmarks & Testing*
<https://magazine.raspberrypi.com/articles/raspberry-pi-3-specs-benchmarks>
- [10] *Virtual Memory Management, Raspberry Pi 3B*
<https://github.com/s-matyukevich/raspberry-pi-os/blob/master/docs/lesson06/rpi-os.md>
- [11] Leon de Boer. *“SmartStart64.S”: A Multicore Bootloader for Raspberry Pi*
<https://github.com/LdB-ECM/Raspberry-Pi-Multicore/blob/master/xRTOS/SmartStart64.S>
- [12] *Bcm2836 Peripherals*
<https://datasheets.raspberrypi.com/bcm2836/bcm2836-peripherals.pdf>

- [13] *Nginx Arhitecture*
<https://natnat1.medium.com/nginx-architecture-c9c93058e8a0>
- [14] *Unikraft Raspberrpi PI Platform Source Code*
<https://github.com/OctavianMihaila/unikraft-rpi>
- [15] *Unikraft Cooperative Scheduler*
<https://github.com/unikraft/unikraft/blob/staging/lib/ukschedcoop/schedcoop.c>

9 ARTIFACTS APPENDIX

9.1 Abstract

In this appendix, instructions on how to run the applications mentioned in this thesis are provided. A detailed description on how to setup Unikraft on Raspberry PI can be found in the appendix of Job's thesis[2] and should be followed before proceeding with the steps presented here. The source code for the SMP capable platform, together with the code for the applications and the build scripts can be found at[14] on the measurements branch.

9.2 Hardware requirements & setup

Raspberry pi 3B

Official Raspberry Pi Power Supply – 5.1 V DC, 2.5 A output, or equivalent

100BASE-TX Ethernet cable (Cat 5e or better)

USB-to-UART (TTL) serial adapter

microSD card

USB microSD card reader/writer

Network Bridge: Configure a network bridge on your host for the Ethernet interface, so that the Raspberry Pi has network access.

Serial connection: (Black - Pin6(GND), White - Pin8(TX), Green - Pin10(RX))

If working on a linux machine, serial can be opened with: `minicom -b 115200 -o -D /dev/ttyUSBx`

After building and writing the unikernel on the sdcard as described below, plug in the power supply and the unikernel will start running.

9.3 Deploy Unikraft

If you've followed the steps in Job's appendix, you now have an SD card preloaded with everything needed to run Unikraft. To swap in a different unikernel, just replace the kernel8.img file in the card's boot partition.

To enable multicore support for the unikernel, open the cmdline.txt file in the boot partition and add the parameter: `maxcpus=X`, where X is the number of cores (1–4) you want the unikernel to use.

For applications that involve network communication, before running the build script in `/unikraft-rpi/measurements/build`, update the client's source code to use the IP address of the machine on which the server is running. The result of running the build script is the creation of a `kernel8.img` that has to be copied on the boot partition of the sdcard, this being the final step before inserting the SD card into the Raspberry Pi and booting the newly built unikernel.